

DSC550Week1

June 8, 2025

```
[19]: # Import Packages required for this session
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
[21]: videogamedf = pd.read_csv("./Video_Games_Sales_as_at_22_Dec_2016.csv")
```

0.1 Display the first ten rows of data.

```
[24]: videogamedf.head(10)
```

```
[24]:
```

	Name	Platform	Year_of_Release	Genre	\
0	Wii Sports	Wii	2006.0	Sports	
1	Super Mario Bros.	NES	1985.0	Platform	
2	Mario Kart Wii	Wii	2008.0	Racing	
3	Wii Sports Resort	Wii	2009.0	Sports	
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	
5	Tetris	GB	1989.0	Puzzle	
6	New Super Mario Bros.	DS	2006.0	Platform	
7	Wii Play	Wii	2006.0	Misc	
8	New Super Mario Bros. Wii	Wii	2009.0	Platform	
9	Duck Hunt	NES	1984.0	Shooter	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	\
0	Nintendo	41.36	28.96	3.77	8.45	82.53	
1	Nintendo	29.08	3.58	6.81	0.77	40.24	
2	Nintendo	15.68	12.76	3.79	3.29	35.52	
3	Nintendo	15.61	10.93	3.28	2.95	32.77	
4	Nintendo	11.27	8.89	10.22	1.00	31.37	
5	Nintendo	23.20	2.26	4.22	0.58	30.26	
6	Nintendo	11.28	9.14	6.50	2.88	29.80	
7	Nintendo	13.96	9.18	2.93	2.84	28.92	
8	Nintendo	14.44	6.94	4.70	2.24	28.32	
9	Nintendo	26.93	0.63	0.28	0.47	28.31	

Critic_Score	Critic_Count	User_Score	User_Count	Developer	Rating
--------------	--------------	------------	------------	-----------	--------

0	76.0	51.0	8	322.0	Nintendo	E
1	NaN	NaN	NaN	NaN	NaN	NaN
2	82.0	73.0	8.3	709.0	Nintendo	E
3	80.0	73.0	8	192.0	Nintendo	E
4	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN
6	89.0	65.0	8.5	431.0	Nintendo	E
7	58.0	41.0	6.6	129.0	Nintendo	E
8	87.0	80.0	8.4	594.0	Nintendo	E
9	NaN	NaN	NaN	NaN	NaN	NaN

0.2 Dimesion on videogame dataframe

```
[27]: nRow, nCol = videogame.df.shape
print(f'There are {nRow} rows and {nCol} columns')
```

There are 16719 rows and 16 columns

0.3 Top five games by critic score.

```
[33]: videogame.df.nlargest(5, 'Critic_Score')
```

```
[33]:
```

	Name	Platform	Year_of_Release	Genre	\
51	Grand Theft Auto IV	X360	2008.0	Action	
57	Grand Theft Auto IV	PS3	2008.0	Action	
227	Tony Hawk's Pro Skater 2	PS	2000.0	Sports	
5350	SoulCalibur	DC	1999.0	Fighting	
16	Grand Theft Auto V	PS3	2013.0	Action	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	\
51	Take-Two Interactive	6.76	3.07	0.14	1.03	
57	Take-Two Interactive	4.76	3.69	0.44	1.61	
227	Activision	3.05	1.41	0.02	0.20	
5350	Namco Bandai Games	0.00	0.00	0.34	0.00	
16	Take-Two Interactive	7.02	9.09	0.98	3.96	

	Global_Sales	Critic_Score	Critic_Count	User_Score	User_Count	\
51	11.01	98.0	86.0	7.9	2951.0	
57	10.50	98.0	64.0	7.5	2833.0	
227	4.68	98.0	19.0	7.7	299.0	
5350	0.34	98.0	24.0	8.8	200.0	
16	21.04	97.0	50.0	8.2	3994.0	

	Developer	Rating
51	Rockstar North	M
57	Rockstar North	M
227	Neversoft Entertainment	T

5350	Namco	T
16	Rockstar North	M

0.4 No of Video Games by Genre

```
[41]: genres = videogamedf['Genre']
genre_counts = genres.value_counts()
print(genre_counts)
```

```
Genre
Action      3370
Sports      2348
Misc        1750
Role-Playing 1500
Shooter     1323
Adventure   1303
Racing      1249
Platform     888
Simulation   874
Fighting     849
Strategy     683
Puzzle       580
Name: count, dtype: int64
```

```
[50]: # Filter for SNES games
snes_games = videogamedf[videogamedf['Platform'] == 'SNES']

# Get the first five games
first_five_snes = snes_games.head(5)

# Print the result
print(first_five_snes)
```

	Name	Platform	Year_of_Release	Genre	\
18	Super Mario World	SNES	1990.0	Platform	
56	Super Mario All-Stars	SNES	1993.0	Platform	
71	Donkey Kong Country	SNES	1994.0	Platform	
76	Super Mario Kart	SNES	1992.0	Racing	
137	Street Fighter II: The World Warrior	SNES	1992.0	Fighting	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	\
18	Nintendo	12.78	3.75	3.54	0.55	20.61	
56	Nintendo	5.99	2.15	2.12	0.29	10.55	
71	Nintendo	4.36	1.71	3.00	0.23	9.30	
76	Nintendo	3.54	1.24	3.81	0.18	8.76	
137	Capcom	2.47	0.83	2.87	0.12	6.30	

Critic_Score	Critic_Count	User_Score	User_Count	Developer	Rating
--------------	--------------	------------	------------	-----------	--------

18	NaN	NaN	NaN	NaN	NaN	NaN
56	NaN	NaN	NaN	NaN	NaN	NaN
71	NaN	NaN	NaN	NaN	NaN	NaN
76	NaN	NaN	NaN	NaN	NaN	NaN
137	NaN	NaN	NaN	NaN	NaN	NaN

0.5 Five publishers with the highest total global sales

```
[52]: # Group by publisher and sum the global sales
publisher_sales = videogame_df.groupby('Publisher')['Global_Sales'].sum()

# Sort in descending order and get the top 5
top_publishers = publisher_sales.sort_values(ascending=False).head(5)

# Print the results
print(top_publishers)
```

```
Publisher
Nintendo          1788.81
Electronic Arts    1116.96
Activision         731.16
Sony Computer Entertainment  606.48
Ubisoft            471.61
Name: Global_Sales, dtype: float64
```

0.6 Calculate the percentage of North American sales

```
[74]: # Calculate the percentage of North American sales
videogame_df['Percentage of Global Sales'] = (videogame_df['NA_Sales'] /
↪videogame_df['Global_Sales']) * 100

# Display the first five rows
df =
↪videogame_df[['Name', 'Platform', 'Year_of_Release', 'Genre', 'NA_Sales', 'Global_Sales', 'Percent
↪of Global Sales']]

print(df.head(5))
```

	Name	Platform	Year_of_Release	Genre	NA_Sales	\
0	Wii Sports	Wii	2006.0	Sports	41.36	
1	Super Mario Bros.	NES	1985.0	Platform	29.08	
2	Mario Kart Wii	Wii	2008.0	Racing	15.68	
3	Wii Sports Resort	Wii	2009.0	Sports	15.61	
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	11.27	

	Global_Sales	Percentage of Global Sales
0	82.53	50.115110
1	40.24	72.266402

2	35.52	44.144144
3	32.77	47.635032
4	31.37	35.926044

0.7 number NaN entries (missing data values) in each column

```
[76]: nan_counts = videogamedf.isnull().sum()
print(nan_counts)
```

```
Name          2
Platform      0
Year_of_Release 269
Genre         2
Publisher     54
NA_Sales      0
EU_Sales      0
JP_Sales      0
Other_Sales   0
Global_Sales  0
Critic_Score  8582
Critic_Count  8582
User_Score    6704
User_Count    9129
Developer     6623
Rating        6769
Percentage of Global Sales  0
dtype: int64
```

```
[83]: # Unique values in 'User Score' Column in dataframe
unique_values = videogamedf['User_Score'].unique()
print(unique_values)
```

```
['8' nan '8.3' '8.5' '6.6' '8.4' '8.6' '7.7' '6.3' '7.4' '8.2' '9' '7.9'
'8.1' '8.7' '7.1' '3.4' '5.3' '4.8' '3.2' '8.9' '6.4' '7.8' '7.5' '2.6'
'7.2' '9.2' '7' '7.3' '4.3' '7.6' '5.7' '5' '9.1' '6.5' 'tbd' '8.8' '6.9'
'9.4' '6.8' '6.1' '6.7' '5.4' '4' '4.9' '4.5' '9.3' '6.2' '4.2' '6' '3.7'
'4.1' '5.8' '5.6' '5.5' '4.4' '4.6' '5.9' '3.9' '3.1' '2.9' '5.2' '3.3'
'4.7' '5.1' '3.5' '2.5' '1.9' '3' '2.7' '2.2' '2' '9.5' '2.1' '3.6' '2.8'
'1.8' '3.8' '0' '1.6' '9.6' '2.4' '1.7' '1.1' '0.3' '1.5' '0.7' '1.2'
'2.3' '0.5' '1.3' '0.2' '0.6' '1.4' '0.9' '1' '9.7']
```

0.8 Median vlaues to fill in for 'NaN' values

```
[85]: user_scores = videogamedf['User_Score']
non_numeric_string = 'tbd'
user_scores = user_scores.replace(non_numeric_string, pd.NA)

# Convert the user score column to numeric
```

```

user_scores = pd.to_numeric(user_scores)

# Calculate the median user score
median_user_score = user_scores.median()

# Replace NaN values with the median
user_scores = user_scores.fillna(median_user_score)

# Update the 'user_score' column in the DataFrame
videogamedf['User_Score'] = user_scores

```

	Name	Platform	Year_of_Release	Genre	Publisher	\
0	Wii Sports	Wii	2006.0	Sports	Nintendo	
1	Super Mario Bros.	NES	1985.0	Platform	Nintendo	
2	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	
3	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	

	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Critic_Score	\
0	41.36	28.96	3.77	8.45	82.53	76.0	
1	29.08	3.58	6.81	0.77	40.24	NaN	
2	15.68	12.76	3.79	3.29	35.52	82.0	
3	15.61	10.93	3.28	2.95	32.77	80.0	
4	11.27	8.89	10.22	1.00	31.37	NaN	

	Critic_Count	User_Score	User_Count	Developer	Rating	\
0	51.0	8.0	322.0	Nintendo	E	
1	NaN	7.5	NaN	NaN	NaN	
2	73.0	8.3	709.0	Nintendo	E	
3	73.0	8.0	192.0	Nintendo	E	
4	NaN	7.5	NaN	NaN	NaN	

	Percentage of Global Sales
0	50.115110
1	72.266402
2	44.144144
3	47.635032
4	35.926044

0.9 Unique values after above Median values calculation

```

[87]: # Unique values in 'User Score' Column in dataframe
unique_values = videogamedf['User_Score'].unique()
print(unique_values)

```

```

[8.  7.5  8.3  8.5  6.6  8.4  8.6  7.7  6.3  7.4  8.2  9.  7.9  8.1  8.7  7.1  3.4  5.3
 4.8  3.2  8.9  6.4  7.8  2.6  7.2  9.2  7.  7.3  4.3  7.6  5.7  5.  9.1  6.5  8.8  6.9
 9.4  6.8  6.1  6.7  5.4  4.  4.9  4.5  9.3  6.2  4.2  6.  3.7  4.1  5.8  5.6  5.5  4.4]

```

```
4.6 5.9 3.9 3.1 2.9 5.2 3.3 4.7 5.1 3.5 2.5 1.9 3.  2.7 2.2 2.  9.5 2.1
3.6 2.8 1.8 3.8 0.  1.6 9.6 2.4 1.7 1.1 0.3 1.5 0.7 1.2 2.3 0.5 1.3 0.2
0.6 1.4 0.9 1.  9.7]
```

[]: